

Below is a comprehensive table of the most common **Convolutional Neural Network (CNN) hyperparameters**. These hyperparameters can be grouped into architecture, training, optimization, and regularization categories.

Category	Hyperparameter	Description	Typical Values
Input	Input Image Size	Width × Height × Channels	28×28×1, 224×224×3
	Batch Size	Number of samples processed together	8, 16, 32, 64, 128, 256
	Data Augmentation	Artificially increases dataset diversity	Flip, Rotation, Crop, Zoom
	Normalization	Scale input pixel values	[0,1], [-1,1], Z-score
Convolution Layer	Number of Convolution Layers	Total convolution layers in the network	2–100+
	Number of Filters	Number of kernels in each layer	16, 32, 64, 128, 256, 512
	Filter (Kernel) Size	Size of convolution kernel	1×1, 3×3, 5×5, 7×7
	Stride	Step size of kernel movement	1, 2
	Padding	Preserve feature map dimensions	Same, Valid
	Dilation Rate	Expands receptive field	1, 2, 4
	Groups	Number of filter groups	1, 2, 4, Depthwise
	Weight Initialization	Initial parameter values	Xavier, He, Random

Category	Hyperparameter	Description	Typical Values
Activation	Bias Initialization	Initial bias values	Zero, Constant
	Activation Function	Non-linearity applied after convolution	ReLU, Leaky ReLU, ELU, GELU, Swish, Sigmoid, Tanh
	Negative Slope	Used in Leaky ReLU	0.01–0.3
Pooling Layer	Pooling Type	Downsampling method	Max, Average, Global Max, Global Average
	Pool Size	Size of pooling window	2×2, 3×3
	Pool Stride	Step size during pooling	2
Normalization	Batch Normalization	Normalize activations	Enabled/Disabled
	Momentum	Running statistics update	0.9–0.99
	Epsilon	Numerical stability constant	1e−5
Fully Connected Layer	Number of Dense Layers	Total dense layers	1–5
	Number of Neurons	Hidden units per dense layer	64, 128, 256, 512, 1024
Output Layer	Output Activation	Final activation	Softmax, Sigmoid, Linear
	Number of Output Units	Depends on task	Number of classes
Loss Function	Loss Function	Objective function	Cross-Entropy, Binary Cross-Entropy, MSE, Hinge
	Label Smoothing	Softens target labels	0–0.2

Category	Hyperparameter	Description	Typical Values
Optimizer	Optimizer Type	Parameter update algorithm	SGD, Adam, RMSprop, AdamW, Adagrad
	Learning Rate	Controls weight updates	1e-5 to 1e-1
	Learning Rate Scheduler	Dynamic LR adjustment	StepLR, Cosine, Exponential, ReduceLROnPlateau
	Momentum	Used in SGD	0.8–0.99
	Beta1	Adam first moment coefficient	0.9
	Beta2	Adam second moment coefficient	0.999
	Epsilon	Numerical stability	1e-8
	Weight Decay	L2 regularization coefficient	1e-6–1e-2
Regularization	Dropout Rate	Fraction of neurons dropped	0.2–0.5
	L1 Regularization	Sparsity penalty	1e-6–1e-3
	L2 Regularization	Weight decay penalty	1e-6–1e-3
	Early Stopping	Stop training when validation stalls	Enabled/Disabled
Training	Number of Epochs	Full passes through training data	10–500
	Shuffle Data	Randomize training samples	True/False
	Gradient Clipping	Prevent exploding gradients	1–10

Category	Hyperparameter	Description	Typical Values
	Mixed Precision Training	Use FP16/FP32	Enabled/Disabled
	Random Seed	Ensures reproducibility	Integer value
	Validation Split	Portion of training data for validation	0.1–0.3
Validation	Cross Validation	K-fold validation	5-fold, 10-fold
	Evaluation Metric	Performance measurement	Accuracy, Precision, Recall, F1-score, AUC, IoU, Dice
Hardware	Device	Training hardware	CPU, GPU, TPU
	Number of Workers	Parallel data loading threads	2–16

Most Important CNN Hyperparameters

The following hyperparameters typically have the greatest impact on model performance:

Rank	Hyperparameter	Importance
1	Learning Rate	Very High
2	Optimizer	Very High
3	Batch Size	Very High
4	Number of Convolution Filters	High
5	Kernel Size	High
6	Number of CNN Layers	High
7	Activation Function	High
8	Dropout Rate	High
9	Weight Decay (L2)	Medium–High
10	Number of Epochs	Medium–High

Rank	Hyperparameter	Importance
11	Pooling Type	Medium
12	Learning Rate Scheduler	Medium
13	Data Augmentation	Medium
14	Weight Initialization	Medium
15	Padding and Stride	Medium

This table covers virtually all hyperparameters encountered in modern CNN architectures, including classic networks (LeNet, AlexNet, VGG), residual networks (ResNet), dense architectures (DenseNet), efficient models (EfficientNet), and contemporary CNN variants.

Is this conversation helpful so far?

